

MML_DAY3

深度学习常用组件

ADAM优化器

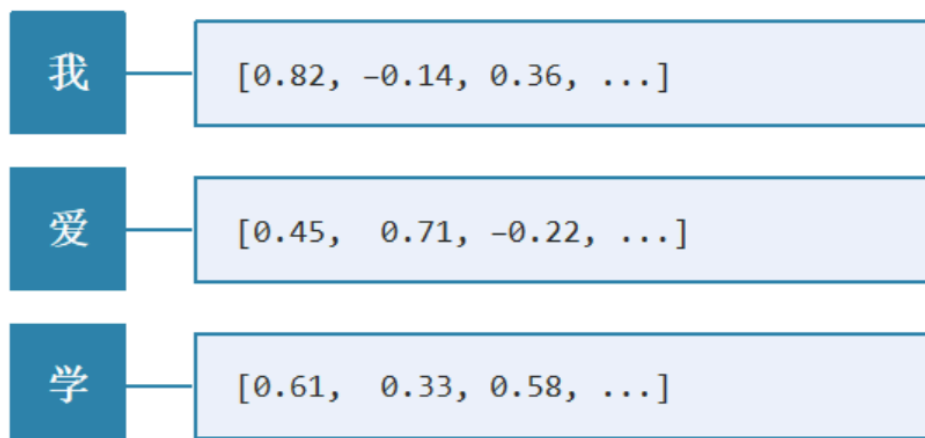
- 计算过程
 - 计算当前梯度: $g_t = \nabla L(\theta_{t-1})$
 - 加入 L2 正则项 (weight_decay) : $g_t = g_t + \lambda \theta_{t-1}$
 - 更新一阶矩 (动量) : $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
 - 更新二阶矩 (梯度平方) : $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
 - 进行偏差修正 (一阶矩) : $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$
 - 进行偏差修正 (二阶矩) : $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
 - 更新参数: $\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
- ADA优化器特点:
 - 加入 L2 正则项:使梯度向0靠近, 防止权重参数过大
 - **一阶矩 (动量)** : 使每一次更新参数都包含历史梯度的信息(历史数据的信息), 越新梯度权重占比越大。增加权重更新的稳定性
 - **二阶矩 (梯度平方)** : 当梯度较大时, 使更新步长自适应变小, 防止梯度爆炸
 - 偏差修正: 随着迭代次数, 更新步长变小, 防止梯度爆炸
- 关键参数:
 - 学习率: lr
 - 一二阶矩参数: β_1, β_2
 - 防止除0小数: ϵ
 - L2正则化参数: λ
 - 使用梯度平方的历史最大值: False/True

Embedding

- 概念：一种将文字映射为向量的组件，一般是固定映射的维度(列数)，根据字数确定行数。优点：有无限组合，解决维度灾难；可以表达字之间的距离关系。

Embedding 解决方案

将每个词映射为固定维度的稠密实数向量



Embedding 优势

- ✓ 低维稠密：通常 64~512 维
- ✓ 语义相似词距离近（可学习）

- Embedding关键参数：
 - 表达一个字符需要的维度：embed_di
 - 字符数量：num_char
 - 特殊字符：pad, unk 用来表示空和未知字符
- 前向传播：
 - 输入：文本字符
 - 根据输入内容进行补齐至seq_len，长度不足补 pad ,过长则截取
 - 根据输入的字符获取对应的索引，根据索引在权重矩阵中获取张量
 - 输出shape：(batch, seq_len, embed_dim)-->(B, L, D)

CNN

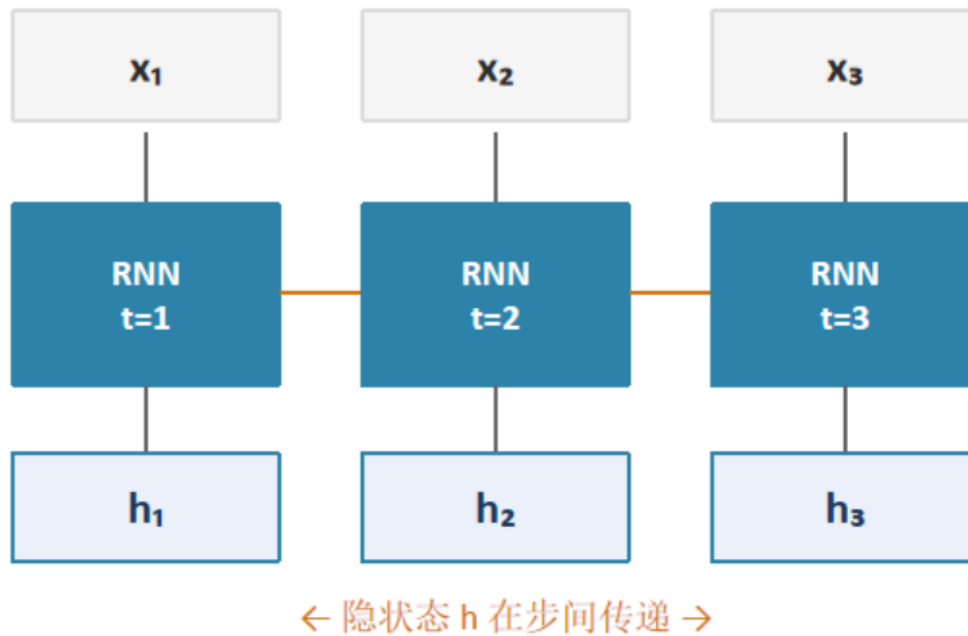
- 概念：卷积层，使用一个卷积核在不同维度上进行滑动，求点积合。图像常用。具有以下特点：
 - 局部感受野：每次只看卷积核大小的内容

- 每个卷积核提取一种特征
- 关键参数：
 - 卷积核大小:kernel_size
 - 步长：每次位移的距离
 - padding：对原数据填充的大小，一般是为了保证输出和输出的维度一致
- 前向传播：一维卷积核为例
 - 输入：(batch, in_channels,in_length)
 - 填充padding
 - 使用卷积核依次划过张量，每次计算点积： $out[i] = \sum_{k=0}^{K-1} w[k] \cdot x[i + k]$ 。每个卷积核，依次计算。
 - **卷积核会覆盖所有通道**，所以一维卷积核实际大小：(in_channels,kernel_size)
 - 输出：(batch, out_channels, out_length)
 - channels_out：等于卷积核数量
 - out_length：由in_length和卷积核参数决定

RNN

- 概念：每一步处理当前 token 时，将上一步的状态 h_{t-1} 充当'记忆'作为输入，在序列中传递信息。
 - 优点：带有历史信息的记忆
 - 缺点：记忆会逐渐模糊和丢失，对应长文本的识别效果差
- 关键参数：
 - input_size:输入参数大小
 - hidden_size：输出参数和隐藏层大小
- 前向传播：单层单向，输入一个文本
 - 输入：(batch, seq_len, input_size) ，input_size就是表示一个字符向量的维度
 - 每个字符循环做： $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$
 - $h_0 = 0$
 - W_h, W_x, b :分别是可训练参数

RNN 序列展开（时间步）



- 输出：(batch, seq_len, hidden_size) , (1, batch, hidden_size)

LSTM

- 概念：在CNN的基础上，添加了遗忘和输出控制策略
- 关键参数：
 - input_size:输入参数大小
 - hidden_size: 输出参数和隐藏层大小
- 前向传播：单层单向，输入一个文本
 - 输入：(batch, seq_len, input_size) , input_size就是表示一个字符向量的维度
 - 计算三个门的值： σ 保障三个门输出是在[0,1]之间的向量，作用就是控制信息权重
 - 遗忘门：决定保留多少历史信息， $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
 - 输入门：决定输入保留多少信息， $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
 - 输出门：决定最后的输出保留多少信息， $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
 - 计算候选值：就是RNN公式， $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b)$
 - 更新候选值：添加历史信息， $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
 - 最后输出： $h_t = o_t \odot \tanh(C_t)$
 - 对每一个字符重复计算...

简单 RNN 的问题

梯度消失：序列很长时，早期信息经多次 $\tanh$ 传递后接近 0，模型「忘记」了很早之前的内容。

LSTM：三个门控制信息流

遗忘门 f_t

决定从细胞状态 c_{t-1} 中丢弃多少旧信息

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

输入门 i_t

决定写入多少新信息到细胞状态

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

输出门 o_t

决定从细胞状态读出多少作为隐状态 h_t

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

细胞状态更新流程

候选值

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b)$$

新的候选信息

细胞更新

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

遗忘旧 + 写入新

隐状态

$$h_t = o_t \odot \tanh(c_t)$$

输出门控制读出量

核心优势

细胞状态 c_t 直接传播，梯度通道更顺畅
→ 有效缓解梯度消失，支持更长序列

- 输出：(batch, seq_len, hidden_size) ， (1, batch, hidden_size)
- 双向传播：训练两个RNN/LSTM，分别以不同的顺序输入，得到双分的输出：(batch, seq_len, hidden_size*2) ， (2, batch, hidden_size)



池化层

- 概念：进行下采样(降维)，一般有Max和Avg两种，降低数据维度，减少计算。注意：池化的结果不受排序影响，所以池化后会**丢失顺序信息**。
- 前行传播：
 - 输入：(B, C, H)，维度H
 - 输出：(B, C)
- 也有非全局池化的方法，类似CNN一样，以窗口的形式，局部池化，不会降维。

归一化层

- 概念：将数据的分布重新归一化到(0,1)分布上，防止梯度爆炸。

- BatchNorm(批次): 图像用多
- LayerNorm(特征维度): NLP用的多
- 前行传播:
 - 输入: (B, C, H)
 - 计算均值, 方差: $\mu_B = \frac{1}{m} \sum x_i, \sigma_B^2 = \frac{1}{m} \sum (x_i - \mu_B)^2$
 - 归一化: $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
 - 仿射变化: $y_i = \gamma \hat{x}_i + \beta$
 - 输出: (B, C, H)

Dropout

- 概念: 训练时随机让一部分沉默(变0), 以此提高网络的鲁棒性。推理时不生效。

训练时



随机 50% 节点被置 0

推理时 (eval mode)



全部节点激活, 输出 $\times (1-p)$ 缩放

